基于神经网络的混凝土抗压强度预测

姓名： 任影儿 **学号：** BC25010012

一、 实验目的与背景

本实验旨在利用神经网络技术解决回归预测问题。所用数据集为真实的混凝土抗压强度数据（共1030个实例）。目标是建立一个深度学习模型，通过输入水泥、高炉矿渣、粉煤灰、水、高效减水剂、粗骨料、细骨料及养护龄期共8个特征参数，准确预测出混凝土的最终抗压强度（MPa）。

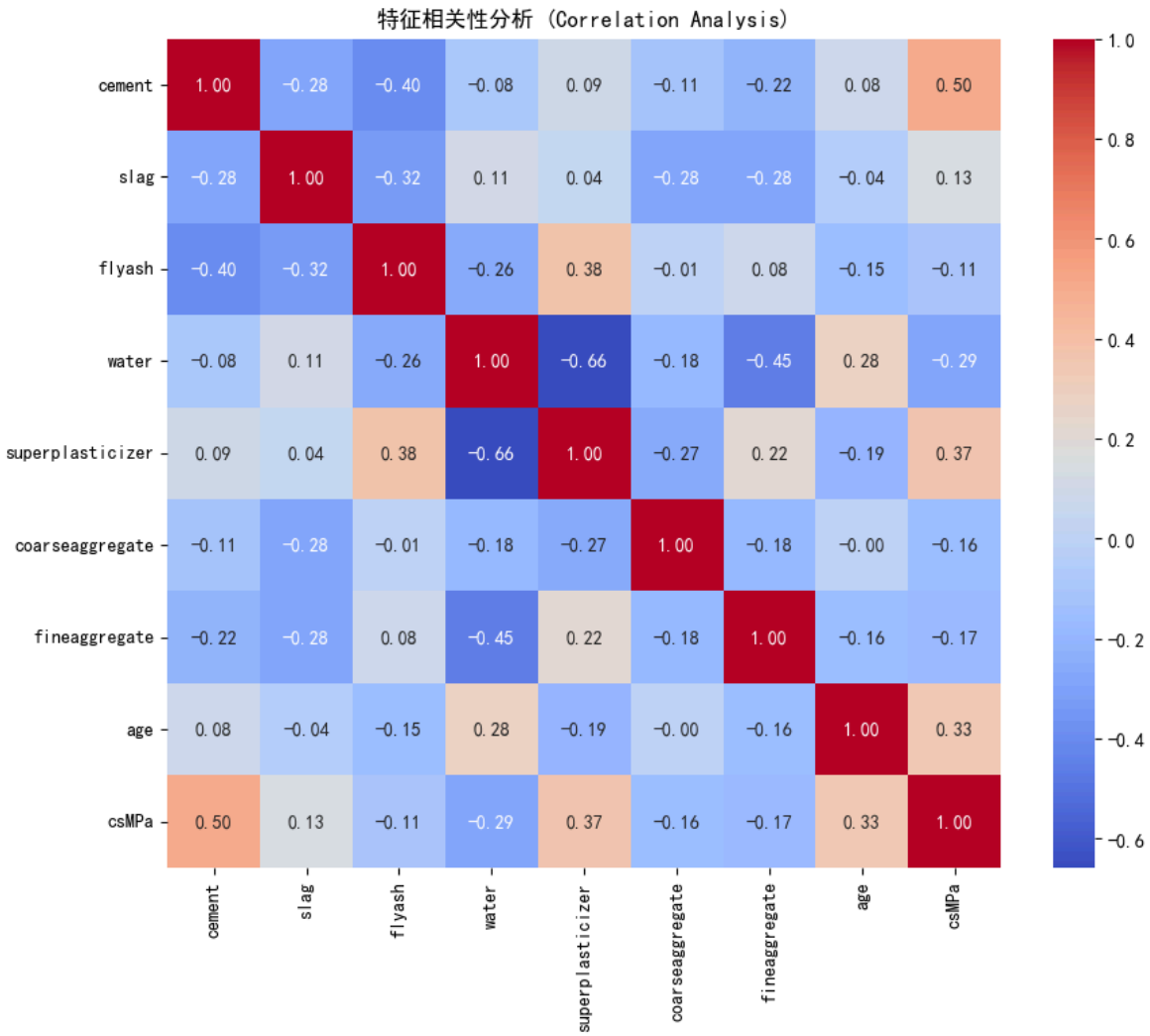
In [1]:
```
import copy
import random

import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from torch.utils.data import DataLoader, TensorDataset

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# 是否启用模型集成
USE_ENSEMBLE = True
ENSEMBLE_SEEDS = [11, 22, 33, 44, 55]
```

二、 数据相关性分析

首先读取混凝土数据集（共1030个实例）。利用皮尔逊相关系数分析特征与抗压强度之间的关系。从生成的热力图可以看出，水泥（Cement）含量与强度呈强正相关，而水（Water）含量与强度呈明显的负相关。

In [2]:
```
file_path = r'e:\神经网络\Concrete_Data_Yeh.csv'
data = pd.read_csv(file_path)

plt.figure(figsize=(10, 8))
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title('特征相关性分析 (Correlation Analysis)')
plt.show()
```



三、 数据预处理

在将数据输入神经网络前，进行如下改进：

1. **数据划分**：先按 8:2 划分训练全集与测试集，再从训练全集中划出验证集（用于调参与早停监控）。
2. **双重标准化**：对输入特征 X 和目标值 y 都做标准化，缓解高低值预测偏差。
3. **小批量训练**：构建 `DataLoader` 进行 mini-batch 训练，使优化更稳定。

In [3]:
```
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values.reshape(-1, 1)

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.2, random_state=SEED
)
print(f"训练集: {X_train.shape[0]} | 验证集: {X_val.shape[0]} | 测试集: {X_test.

x_scaler = StandardScaler()
y_scaler = StandardScaler()
```

```
X_train_scaled = x_scaler.fit_transform(X_train)
X_val_scaled = x_scaler.transform(X_val)
X_test_scaled = x_scaler.transform(X_test)

y_train_scaled = y_scaler.fit_transform(y_train)
y_val_scaled = y_scaler.transform(y_val)

X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32)
X_val_tensor = torch.tensor(X_val_scaled, dtype=torch.float32)
y_val_tensor = torch.tensor(y_val_scaled, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)

train_loader = DataLoader(
    TensorDataset(X_train_tensor, y_train_tensor), batch_size=32, shuffle=True
)
```

```
训练集: 659 | 验证集: 165 | 测试集: 206
```

四、 神经网络设计与模型训练

本实验将混凝土强度预测视为非线性回归问题，采用多层感知机（MLP）进行建模。相较于线性回归，MLP 能通过非线性激活函数学习不同特征与强度之间更复杂的耦合关系。

4.1 网络结构设计

模型采用三层全连接结构：$8 \to 192 \to 96 \to 1$。

- 输入层 8 个神经元，对应 8 个原始特征；
- 两个隐藏层分别为 192 与 96 个神经元，并使用 `ReLU` 激活函数增强非线性表达能力；
- 输出层为 1 个神经元，输出连续值（抗压强度）。

4.2 优化器与损失函数

- 损失函数选用均方误差（MSE），其形式为：

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

  MSE 对较大误差更敏感，适合本任务中对偏差样本进行更强约束。

- 优化器使用 `AdamW`，相较于标准 Adam，权重衰减与梯度更新解耦，可在保持收敛速度的同时缓解过拟合。

4.3 训练策略

为了提高泛化能力并避免训练后期震荡，本实验引入以下策略：

1. **Mini-batch 训练**：使用 `DataLoader(batch_size=32)` 进行小批量梯度更新，提高优化稳定性；

2. **学习率自适应调度**：使用 `ReduceLROnPlateau`，当验证集损失在若干轮内无明显下降时自动减小学习率；
3. **早停机制**：当验证集损失长期不提升时提前终止训练，并回滚到验证集表现最优的参数；
4. **验证集监控**：训练过程中同时记录训练损失与验证损失，用于判断是否出现过拟合。

4.4 集成学习增强

单个神经网络受随机初始化与批次顺序影响，结果会有波动。为降低方差、提高预测稳定性，本实验采用多随机种子训练多个子模型（seed=11/22/33/44/55），并对测试集预测结果取平均：

$$\hat{y}_{ensemble} = \frac{1}{K}\sum_{k=1}^{K}\hat{y}^{(k)}$$

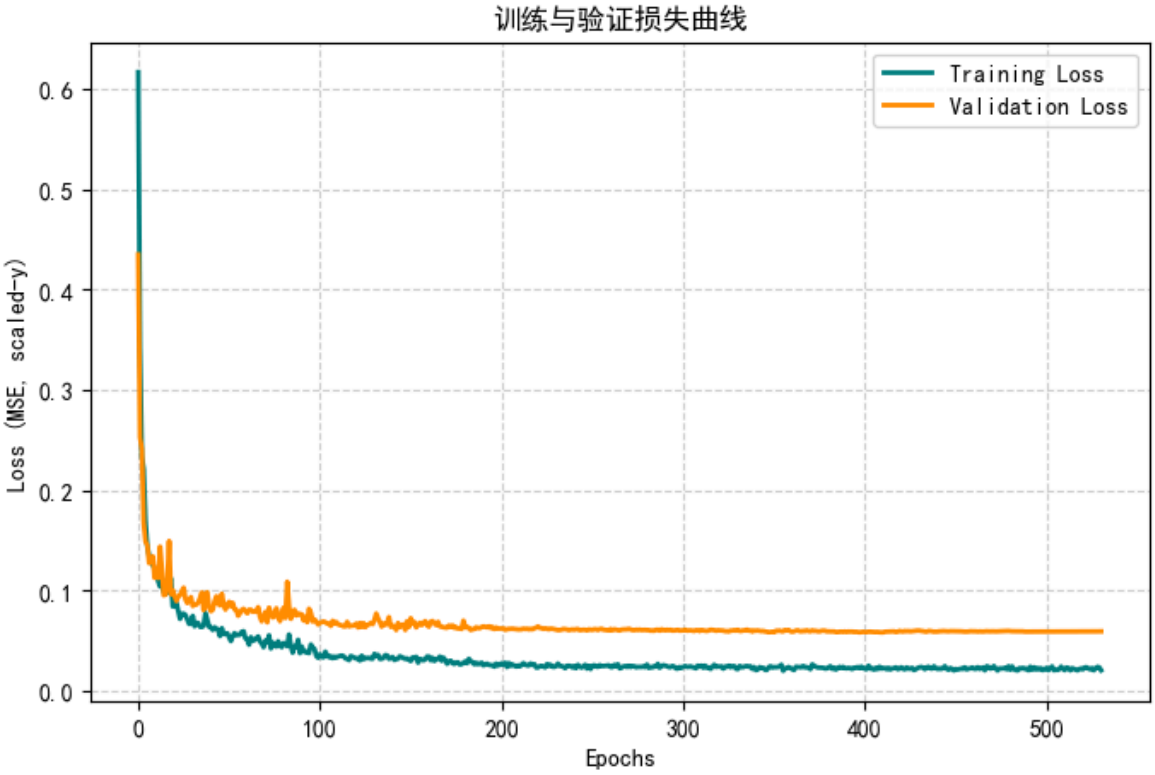
其中 $K = 5$。该策略通常可在不显著增加模型复杂度的前提下提升整体精度与鲁棒性。

In [4]:
```
class ConcreteStrengthNet(nn.Module):
    def __init__(self, input_dim, hidden1=192, hidden2=96, dropout=0.05):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden1),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Dropout(dropout / 2.0),
            nn.Linear(hidden2, 1),
        )

    def forward(self, x):
        return self.net(x)


def train_one_model(seed, verbose=False):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

    model = ConcreteStrengthNet(input_dim=X.shape[1])
    criterion = nn.MSELoss()
    optimizer = optim.AdamW(model.parameters(), lr=0.002, weight_decay=1e-6)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=25, min_lr=5e-6
    )

    max_epochs = 1200
    early_stop_patience = 120
    best_val_loss = float('inf')
    best_state = None
    no_improve = 0

    train_losses = []
    val_losses = []
```

```
    for epoch in range(max_epochs):
        model.train()
        running_loss = 0.0

        for xb, yb in train_loader:
            pred = model(xb)
            loss = criterion(pred, yb)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * xb.size(0)

        train_loss = running_loss / len(train_loader.dataset)

        model.eval()
        with torch.no_grad():
            val_pred = model(X_val_tensor)
            val_loss = criterion(val_pred, y_val_tensor).item()

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        scheduler.step(val_loss)

        if val_loss < best_val_loss - 1e-7:
            best_val_loss = val_loss
            best_state = copy.deepcopy(model.state_dict())
            no_improve = 0
        else:
            no_improve += 1

        if verbose and (epoch + 1) % 100 == 0:
            lr_now = optimizer.param_groups[0]['lr']
            print(f"Epoch [{epoch + 1}/{max_epochs}] | Train: {train_loss:.4f} |

        if no_improve >= early_stop_patience:
            if verbose:
                print(f"早停触发：在第 {epoch + 1} 轮停止训练。")
            break

    if best_state is not None:
        model.load_state_dict(best_state)

    with torch.no_grad():
        pred_scaled = model(X_test_tensor).cpu().numpy()
    pred = y_scaler.inverse_transform(pred_scaled).reshape(-1)

    return pred, train_losses, val_losses, best_val_loss


USE_ENSEMBLE = True
ENSEMBLE_SEEDS = [11, 22, 33, 44, 55]

if USE_ENSEMBLE:
    print('开始训练（集成模式）...')
    pred_list = []
    train_losses, val_losses = None, None
    for i, seed in enumerate(ENSEMBLE_SEEDS):
        pred_i, tr_loss, va_loss, best_val = train_one_model(seed=seed, verbose=
        pred_list.append(pred_i)
        print(f"子模型 seed={seed} 最佳验证损失: {best_val:.5f}")
```

```
        if train_losses is None:
            train_losses, val_losses = tr_loss, va_loss
    predictions_np = np.mean(np.vstack(pred_list), axis=0)
else:
    print('开始训练（单模型模式）...')
    predictions_np, train_losses, val_losses, _ = train_one_model(seed=SEED, ver

plt.figure(figsize=(8, 5))
plt.plot(train_losses, label='Training Loss', color='teal', linewidth=2)
plt.plot(val_losses, label='Validation Loss', color='darkorange', linewidth=2)
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE, scaled-y)')
plt.title('训练与验证损失曲线')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
开始训练（集成模式）...
Epoch [100/1200] | Train: 0.0329 | Val: 0.0661 | LR: 0.001000
Epoch [200/1200] | Train: 0.0264 | Val: 0.0620 | LR: 0.000250
Epoch [300/1200] | Train: 0.0229 | Val: 0.0601 | LR: 0.000125
Epoch [400/1200] | Train: 0.0231 | Val: 0.0588 | LR: 0.000063
Epoch [500/1200] | Train: 0.0216 | Val: 0.0592 | LR: 0.000008
早停触发：在第 531 轮停止训练。
子模型 seed=11 最佳验证损失: 0.05823
子模型 seed=22 最佳验证损失: 0.06020
子模型 seed=33 最佳验证损失: 0.06532
子模型 seed=44 最佳验证损失: 0.05721
子模型 seed=55 最佳验证损失: 0.04756
```



五、 模型评估与结果可视化

在测试集上使用 MSE、MAE、R^2 三个指标综合评估模型性能：

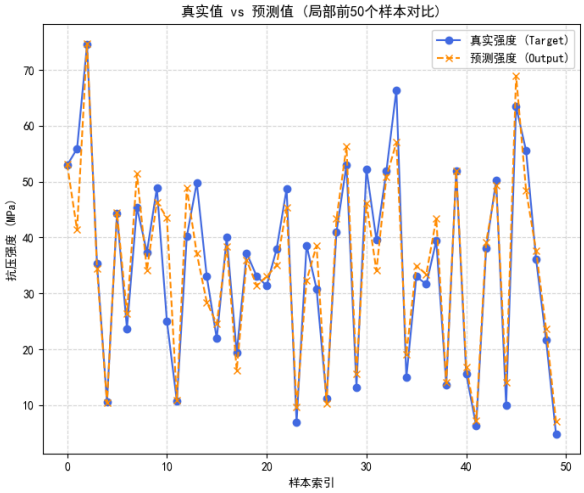
- **MSE**：反映误差平方的平均水平，越小越好。

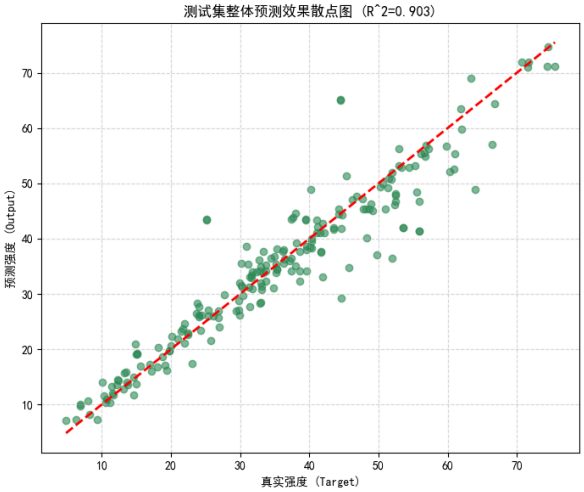
- **MAE**：反映平均绝对误差，单位为 MPa，更直观。
- R^2：反映拟合优度，越接近 1 越好。

此外，统计了绝对误差超过 8 MPa 的样本占比，用于描述离散点情况。

In [ ]:
```
y_test_np = y_test.reshape(-1)

mse = mean_squared_error(y_test_np, predictions_np)
mae = mean_absolute_error(y_test_np, predictions_np)
r2 = r2_score(y_test_np, predictions_np)
residual = predictions_np - y_test_np
outlier_ratio = (np.abs(residual) > 8.0).mean() * 100

print(f"测试集 MSE: {mse:.4f}")
print(f"测试集 MAE: {mae:.4f} MPa")
print(f"测试集 R^2: {r2:.4f}")
print(f"|误差| > 8 MPa 的样本占比: {outlier_ratio:.2f}%\n")

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(y_test_np[:50], label='真实强度 (Target)', color='royalblue', marker='o
plt.plot(predictions_np[:50], label='预测强度 (Output)', color='darkorange', lin
plt.title('真实值 vs 预测值 (局部前50个样本对比)')
plt.xlabel('样本索引')
plt.ylabel('抗压强度 (MPa)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)

plt.subplot(1, 2, 2)
plt.scatter(y_test_np, predictions_np, alpha=0.6, color='seagreen')
line_min = min(y_test_np.min(), predictions_np.min())
line_max = max(y_test_np.max(), predictions_np.max())
plt.plot([line_min, line_max], [line_min, line_max], 'r--', linewidth=2)
plt.xlabel('真实强度 (Target)')
plt.ylabel('预测强度 (Output)')
plt.title(f'测试集整体预测效果散点图 (R^2={r2:.3f})')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

```
测试集 MSE: 24.9396
测试集 MAE: 3.1924 MPa
测试集 R^2: 0.9032
|误差| > 8 MPa 的样本占比: 9.71%
```





模型整体拟合能力强、平均误差低，已经达到较好的预测水平，仅在少量极端样本上存在偏差。表明模型在少量复杂或极端样本上仍有改进空间，但不影响整体预测性能。